



Exercice d'introduction à la technique de programmation informatique ORM

Exercice d'introduction à la technique de programmation informatique ORM

I. SQLAlchemy

I.I Qu'est-ce que SQLAlchemy ?

SQLAlchemy est une bibliothèque Python qui permet, comme `Psycopg`, de se connecter et d'interagir avec des bases de données. La différence fondamentale entre les deux réside dans le fait que SQLAlchemy permet d'interagir avec n'importe quelle base de données et pas seulement PostgreSQL. Plus précisément, SQLAlchemy permet de se connecter aux bases de données suivantes : `SQLite`, `PostgreSQL`, `MySQL` & `MariaDB`, `Oracle`, et `Microsoft SQL Server`.

I.II Pourquoi passer par SQLAlchemy ?

L'avantage principal de SQLAlchemy par rapport à `Psycopg` est sa capacité à pouvoir généraliser du code Python (et même SQL) à différentes bases de données rendant ainsi celui-ci agnostique à la base de données utilisée. Vous n'aurez alors plus besoin d'apprendre la syntaxe Python d'un connecteur particulier mais seulement celle de SQLAlchemy.

I.III Différences de langage SQL

Certaines bases de données cependant n'utilisent pas les mêmes syntaxes SQL. Imaginez par exemple que vous voulez sélectionner des utilisateurs dans une table et créer au passage une colonne décrivant si votre utilisateur est majeur ou non.

Dans ce cas dans PostgreSQL (référez vous au Notebook `SQL - Langage Partie 4` du cours `SQL - Implémentation d'une base de données relationnelle (FR)` au chapitre 8.3. `Clause CASE`) la syntaxe SQL sera :

```
1 | SELECT user, CASE WHEN age > 18 THEN 'Adulte' ELSE 'Mineur' END FROM users;
```

Pourtant, en utilisant `MySQL` la syntaxe SQL ressemblera plutôt à :

```
1 | SELECT user, IF(age > 18, 'Adulte', 'Mineur') FROM users;
```

On comprend alors que la syntaxe est extrêmement dépendante de la base de données et que donc on ne peut pas simplement passer d'un script écrit avec SQLAlchemy d'une base de données à une autre en changeant uniquement le connecteur.

C'est là qu'intervient `ORM`.

II. Object-Relational Mapping

II.I Introduction

L'`ORM` est une méthode rendant encore plus transparente l'utilisation de certaines bibliothèques comme SQLAlchemy permettant de se connecter à diverses bases de données. En effet, cette méthode repose sur une syntaxe particulière qui va vous permettre de généraliser votre script à n'importe quelle base de données en changeant simplement le connecteur.

Nous allons donc voir dans ce Notebook comment fonctionne cette méthode et comment l'utiliser en pratique.

II.II Démarrage et connexion à PostgreSQL

Avant d'exploiter les particularités de `ORM` il nous faut instancier une base de données. Par habitude avec le cours nous utiliserons une base de données PostgreSQL.

Créer un fichier `compose.yaml` avec le contenu du bloc suivant et exécuter celui-ci.

```
1 | version: '3.8'
2 | services:
3 |   db:
4 |     container_name: pg_container
5 |     image: postgres:16-alpine
6 |     restart: always
7 |     environment:
8 |       POSTGRES_USER: daniel
9 |       POSTGRES_PASSWORD: datascientest
10 |      POSTGRES_DB: dst_db
11 |     ports:
12 |       - "5432:5432"
13 |   pgadmin:
14 |     container_name: pgadmin4_container
15 |     image: dpage/pgadmin4
16 |     restart: always
17 |     environment:
18 |       PGADMIN_DEFAULT_EMAIL: daniel@datascientest.com
19 |       PGADMIN_DEFAULT_PASSWORD: data_engineer
20 |     ports:
21 |       - "5050:80"
```

Vous disposez maintenant d'une base de données à laquelle se connecter. Pour s'y connecter avec SQLAlchemy il faudra d'abord installer ce module ainsi que `Psycopg` (qui est utilisé par SQLAlchemy pour se connecter à une base de données PostgreSQL) :

```
1 | pip install sqlalchemy psycopg[binary]
```

Nous pouvons maintenant nous y connecter facilement à l'aide du script suivant :

```
1 | from sqlalchemy import create_engine, text
2 |
3 | # Connexion avec psycopg 3
4 | engine = create_engine("postgresql+psycopg://daniel:datascientest@localhost:5432/dst_db")
5 |
6 | # Exemple de requête simple
7 | with engine.connect() as conn:
8 |     result = conn.execute(text("SELECT version();"))
9 |     print(result.fetchone())
```

Ici, remarquez la syntaxe du connecteur que vous pourrez réutiliser par la suite, on spécifie le nom d'utilisateur, le mot de passe, le port et la database à utiliser. Si tout se passe comme prévu vous devriez alors avoir un résultat ressemblant plus ou moins à celui-ci :

```
1 | ['PostgreSQL 16.9 on x86_64-pc-linux-musl, compiled by gcc (Alpine 14.2.0) 14.2.0, 64-bit',]
```

Maintenant que nous sommes bien connectés. Nous pouvons passer au cœur du sujet.

II.III Interagir avec des tables via la méthode ORM

La première chose à savoir avec la méthode `ORM` est que le `O` de `ORM` signifie objet (on le savait déjà) mais qu'est ce qu'un objet en Python ?

Exactement, un objet est une CLASSE. Donc que veut dire `ORM` ? Et bien simplement que nous allons interagir avec la base de données en utilisant des classes et non plus en exécutant du code SQL directement sur celle-ci.

Pour créer une classe nous avons besoin de deux éléments en Python :

- la base : l'élément qui va créer les tables.
- l'objet : la classe que l'on va utiliser pour définir notre table.

La base est simple à définir. En effet, pour se faire il suffit d'importer `declarative_base` de `sqlalchemy.orm` et d'instancier un objet `declarative_base` :

```
1 | from sqlalchemy.orm import declarative_base
2 |
3 | Base = declarative_base()
```

Maintenant que nous avons notre base, il ne reste plus qu'à définir l'objet qui va représenter la table. Ici on va devoir importer quelques éléments supplémentaires notamment pour définir les types des colonnes de notre table. Ces types se situent directement la librairie `sqlalchemy`.

On y retrouve notamment :

- `Column` : pour désigner une colonne de notre table.
- `Integer` : pour définir un entier standard.
- `String` : pour définir une chaîne de caractères.

Il existe bien d'autres types, vous pourrez les retrouver ici :

[TYPES](#)

Il faut définir le nom de la table non pas avec le nom de la classe mais avec l'attribut `__tablename__`.

Enfin, on peut définir une colonne comme étant la clé primaire grâce au paramètre `primary_key`.

Finalement, le code pour définir une table va donc ressembler au suivant :

```
1 | from sqlalchemy import Column, Integer, String
2 |
3 | class User(Base):
4 |     __tablename__ = 'users'
5 |     id = Column(Integer, primary_key=True)
6 |     user = Column(String)
```

Pour créer la table il suffira alors d'appeler la méthode `create_all` du sous-objet `Base.metadata` en lui passant comme paramètre `engine` (défini pour se connecter à la base de données) :

```
1 | Base.metadata.create_all(engine)
```

Quelques paramètres pourraient vous intéresser ici. Si vous voulez créer une table en particulier, alors il faudra la passer dans une liste du paramètre `tables`.

Attention, il ne faudra pas passer la table directement mais son attribut `__table__`.

Par exemple si vous voulez créer uniquement la table `users` :

```
1 | Base.metadata.create_all(engine, tables=[User.__table__])
```

De même, si vous exécutez cette méthode en l'état, si la table existe déjà alors l'instruction renverra une erreur. Pour éviter ce comportement il faudra spécifier le paramètre `checkfirst=True` (équivalent à un `CREATE TABLE IF NOT EXISTS`).

Si vous ne voulez pas créer mais supprimer alors il suffit de remplacer `create_all` par `drop_all` (qui prend les mêmes paramètres en entrée).

II.IV Quelques requêtes avec la méthode ORM

Maintenant que notre table est prête à l'emploi, nous pouvons la populer avec quelques exemples. Pour se faire vous aurez besoin de l'élément `sessionmaker` de `sqlalchemy.orm` afin d'initialiser une session :

```
1 | from sqlalchemy.orm import sessionmaker
2 |
3 | Session = sessionmaker(bind=engine)
4 | session = Session()
```

Il est maintenant possible de créer un user très facilement en utilisant notre objet `User` :

```
1 | new_user = User(id=37, user='Nintri')
```

Pour l'ajouter à la table il suffira d'utiliser la méthode `add` de notre session avec notre élément et ensuite valider l'insertion avec la méthode `commit` (un peu comme `git`) :

```
1 | session.add(new_user)
2 | session.commit()
```

Pour récupérer les éléments de la table `users` il vous suffira alors d'écrire une requête avec l'objet `session` en passant par la méthode `query` avec pour paramètre l'objet représentant la table que vous voulez requêter.

Pour en obtenir tous les éléments il vous suffira d'y accoler encore après la méthode `all` :

```
1 | users = session.query(User).all()
2 | for user in users:
3 |     print(user.id, user.user)
```

Vous devez alors observer la sortie suivante :

```
1 | 37 Nintri
```

Comme vous le constatez, vous n'avez pas utilisé le langage SQL, tout s'est fait de façon transparente avec SQLAlchemy et est donc transférable très facilement à d'autres bases de données.

II.V Quelques tests et quelques commandes intéressantes

Le premier test que nous allons faire est simplement celui de la création d'une table existante déjà avec le paramètre `checkfirst=True` pour prouver que cela ne génère pas d'erreur :

```
1 | Base.metadata.create_all(engine, checkfirst=True)
2 | users = session.query(User).all()
3 | for user in users:
4 |     print(user.id, user.user)
```

La sortie de ce bloc de code est la suivante :

```
1 | 37 Nintri
```

On a bien notre donnée et aucune erreur.

Maintenant faisons ça :

```
1 | Base.metadata.create_all(engine, checkfirst=False)
```

On obtient instantanément une erreur, cela évite les potentiels conflits dans votre code Python.

Rajoutons une donnée sans spécifier l'id :

```
1 | new_user = User(user='Emperon')
2 | session.add(new_user)
3 | session.commit()
4 | users = session.query(User).all()
5 | for user in users:
6 |     print(user.id, user.user)
```

On constate que le code se lance convenablement et que SQLAlchemy a décidé tout seul d'attribuer l'id 1 à notre nouvelle ligne.

Cela vous évite à vous posez la question de l'unicité de la variable `id`. Il existe d'ailleurs des façons de spécifier une génération particulière d'id mais nous ne les verrons pas ici.

Avant de passer à la section suivante nous allons voir une dernière opération de requête intéressante : le filtre.

Pas de présentation, directement une démonstration :

```
1 | all_users = session.query(User).all()
2 | user_1 = session.query(User).filter_by(id=1).all()
3 | for user in user_1:
4 |     print(user.id, user.user)
```

Directement on a filtré ici la ligne dont `id` est 1. Il existe bien d'autre opération que la simple requête et le filtre. Non nécessaire pour l'examen, vous les retrouverez cependant ici : <https://docs.sqlalchemy.org/en/14/orm/query.html>.

Relations entre tables

Clés étrangères

La dernière chose que nous verrons dans ce cours et qui vous servira pour l'examen est évidemment la syntaxe pour créer des liens entre les tables car nous travaillons avant tout avec des bases de données RELATIONNELLES il faut donc des RELATIONS pour rendre ça utile et consistant.

Pour créer une relation entre deux tables il suffit de créer un attribut `relationship` dans la table de votre choix ainsi qu'une clé étrangère. Si on créé une table `addresses` par exemple sa déclaration sera comme suit :

```
1 | from sqlalchemy import ForeignKey
2 | from sqlalchemy.orm import relationship
3 |
4 | class Address(Base):
5 |     __tablename__ = 'addresses'
6 |     email = Column(Integer, primary_key=True)
7 |     email = Column(String)
8 |     user_id = Column(Integer, ForeignKey('users.id'))
9 |     user = relationship("User")
10 |
11 | Base.metadata.create_all(engine, checkfirst=True)
```

Vous pouvez alors insérer des données comme suit :

```
1 | u = User(user='Nintri')
2 | session.add(u)
3 | session.commit()
4 |
5 | a = Address(email='nintri@example.com', user=u)
6 | session.add(a)
7 | session.commit()
```

Ici on commence par créer le `user` car c'est celui qui est relié dans `addresses`. Puis on commit et enfin vous pourrez créer votre adresse en spécifiant le user relié à celle-ci.

On peut vérifier que la clé étrangère dans `addresses` est alors bien renseignée :

```
1 | all_users = session.query(User).all()
2 | for user in all_users:
3 |     print(user.id, user.user)
4 |
5 | all_addresses = session.query(Address).all()
6 | for address in all_addresses:
7 |     print(address.id, address.email, address.user_id)
```

Si vous avez déjà créé une classe représentant une table alors vous pouvez étendre la définition précédente avec l'ajout de l'attribut :

```
1 | __table_args__ = {'extend_existing': True}
```

Conclusion

Ce Notebook permet de voir dans un temps assez court comment vous pouvez utiliser les classes de Python afin d'efficacement interagir avec la base de données de votre choix. Vous avez là tous les éléments nécessaires pour passer l'examen.

Il est recommandé de prendre la main sur ces différentes notions avant de passer l'examen (même si la documentation vous sera toujours accessible, il est question ici de gagner du temps).

Cours terminé ?

✓ Marquer comme terminé